



Introducción a React

{desafío}
latam_

React

Unidad	Clases (sincrónico)	Autoaprendizaje (asincrónico)	Tutoría (sincrónico)
Introducción a React	2 horas	Desde 6 horas	2 horas
Estados de los componentes y eventos	2 horas	Desde 6 horas	2 horas
Renderización dinámica de componentes	2 horas	Desde 6 horas	2 horas
Consumo de APIs con React	2 horas	Desde 6 horas	2 horas
React Router I	2 horas	Desde 6 horas	2 horas
Context	2 horas	Desde 6 horas	2 horas
React Router II	2 horas	Desde 6 horas	2 horas
JWT	2 horas	Desde 6 horas	2 horas
Prueba	0 horas	Desde 6 horas	0 horas
Receso	0 horas	0 horas	0 horas



**Activen las cámaras los que puedan y
pasemos asistencia**

Crear aplicaciones web en React que muestren información, a través de componentes y hagan uso de la sintaxis JSX.

- Unidad 1: Introducción a React.
- Unidad 2: Estados de los componentes y eventos.
- Unidad 3: Renderización dinámica de componentes.
- Unidad 4: Consumo de APIs con React.
- Unidad 5: React Router I
- Unidad 6: Context
- Unidad 7: React Router II
- Unidad 8: JWT



Te encuentras aquí





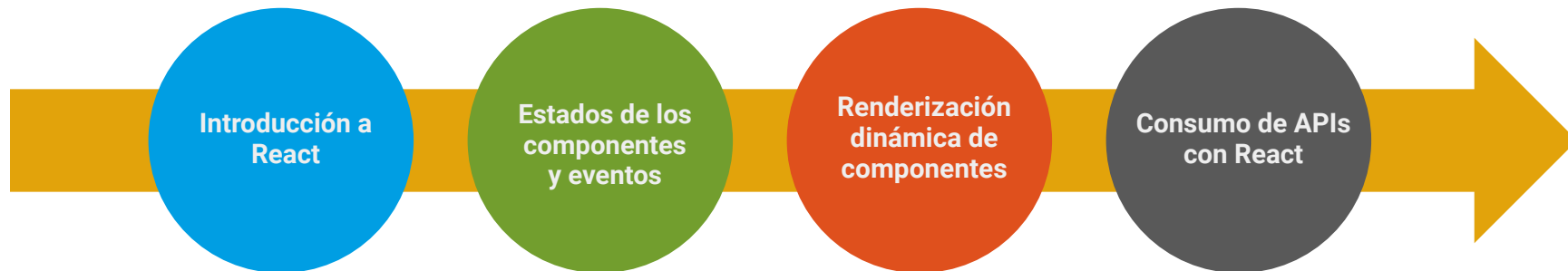
Inicio

{**desafío**}

latam_



Contexto del módulo



Unidad 1:

Crear y reutilizar componentes funcionales en React

Unidad 2:

Utilizar useState y eventos para cambiar estados de una componente en React

Unidad 3:

Crear dinámicamente múltiples componentes partir de una estructura de datos, arreglos, objetos y JSON

Unidad 4:

Utilizar fetch y hooks para mostrar información proveniente de terceros en componentes de React

`/*` Crear aplicaciones web en React que muestren información, a través de componentes y hagan uso de la sintaxis JSX. `*/`

**Objetivo de
la unidad**

`/* Crear una aplicación y un componente utilizando React */`

`/* Crear una componente que cargue otro componente */`

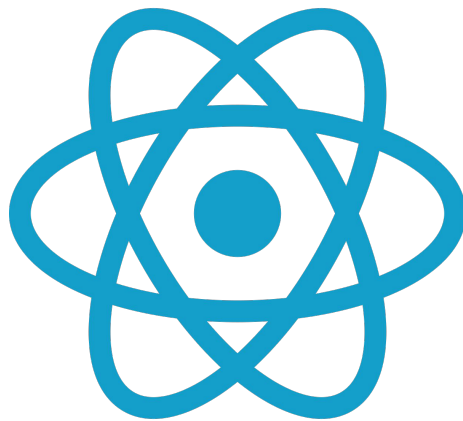
`/* Crear un componente que cargue múltiples componentes */`

`/* Pasar props a un componente */`

Objetivos

¿Por qué aprender React?

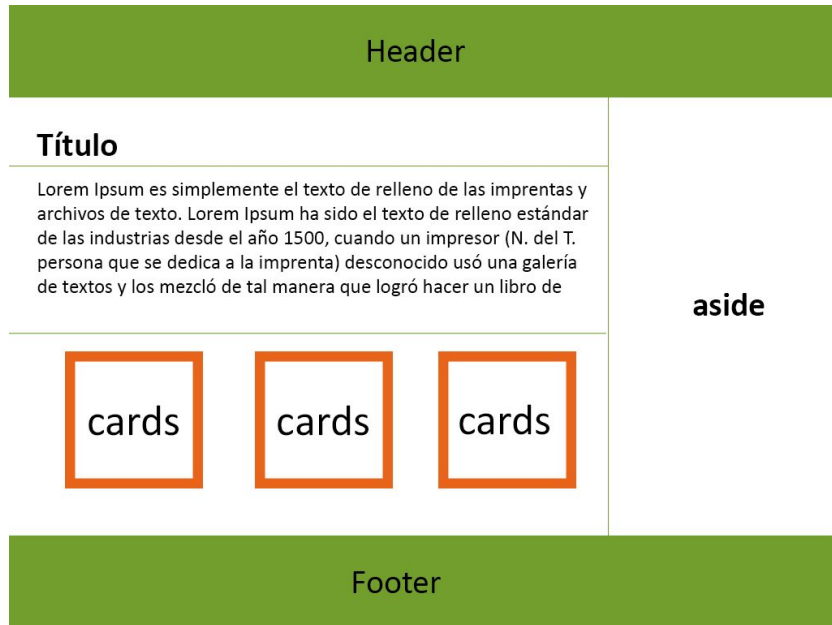
¿Por qué React?



- Es una biblioteca que permite desarrollar en términos de componentes reutilizables.
- Un componente consiste en uno o más elementos HTML que funcionan como un todo.
- Es una de las tecnologías con mayor demanda en la actualidad y se ha mantenido así por años.
- Permite crear aplicaciones **web** y **móviles**.
 - En este curso solo abordaremos aplicaciones web.

¿Qué es un componente?

- Los componentes en React son elementos que construimos para mostrarlos en el navegador.
- Separar nuestra aplicación en componentes, nos permite crear código reutilizable y manejar los comportamientos de manera aislada.
- En la imagen, uno o un conjunto de los elementos podrían ser una componente.





Desarrollo

{desafío}
latam_



¿Qué herramientas necesitamos para empezar?

Visual Studio Code

En este curso ocuparemos el editor de código [Visual Studio Code](#).



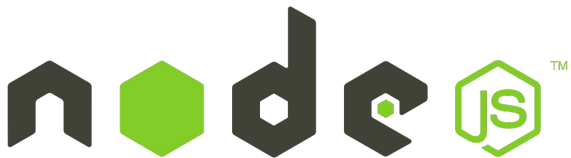
Un navegador

Para esta clase utilizaremos navegador [Google Chrome](#).



Node JS

En este curso ocuparemos node JS que es un entorno de ejecución de JavaScript en el servidor.

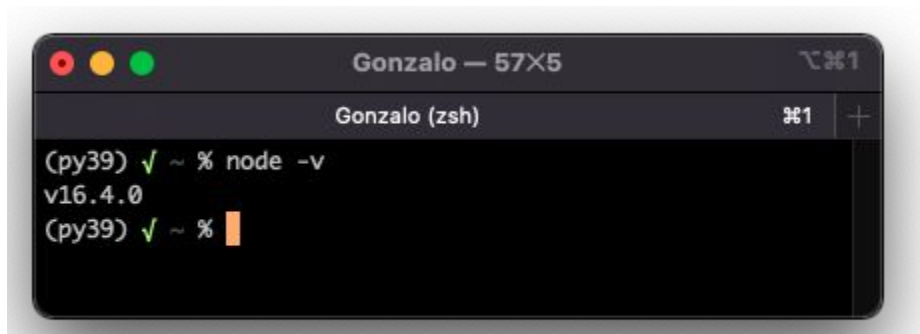


Gestor de paquetes

Npm se instala automáticamente junto a Node. Utilizaremos además el gestor de paquetes npm para instalar dependencias en nuestras aplicaciones de react.



Instalamos node

A terminal window titled 'Gonzalo — 57X5' with a dark background. The window shows a prompt '(py39) ✓ ~ %' followed by the command 'node -v'. The output is 'v16.4.0'. The prompt is followed by a cursor. The window has standard macOS window controls (red, yellow, green buttons) and a zoom icon in the top right corner.

```
(py39) ✓ ~ % node -v
v16.4.0
(py39) ✓ ~ %
```

- Si no tienes instalado node ingresa a:
<https://nodejs.org/en/download/>
- Para actualizar en linux y osx: `npm install -g npm@latest`
- Verifiquemos la instalación

Vite

Para crear nuestros proyectos en react utilizaremos Vite.

Vite es una herramienta de construcción de aplicaciones web que ayudará a mejorar el rendimiento de proyectos basados en JavaScript, sobre todo aquellos que utilizan frameworks o librerías como React.



Creemos nuestra primera aplicación en React

Ejercicio

Realicemos un ejercicio guiado:

1. Crear un directorio con el nombre **Módulo - React I**.
2. Abrir el directorio en la consola o terminal de comandos.
3. Para crear el proyecto, utilizaremos Vite.
 - a. Ejecutar el comando: `npm create vite`.
 - b. Al ejecutar este comando instalaremos Vite.

Demostración



Ejercicio

Realicemos un ejercicio guiado:

4. Le asignamos el nombre **hola-mundo-react** al proyecto.
5. Seleccionamos la opción React y luego JavaScript.
6. Accedemos al directorio **/hola-mundo-react** desde el editor de texto.
7. Levantamos la aplicación en el servidor local con el comando **npm run dev**.
8. Observamos en el navegador la información que se muestra.

Demostración



Estructura de carpetas importantes de React

Carpeta /nodemodules

Dentro de esta carpeta estarán todas las dependencias internas y propias de react para que funcione.

Además, se almacenarán aquellas otras dependencias que vayamos instalando a lo largo del curso.

Carpeta /public

Con Vite, la carpeta public almacena los archivos estáticos que se deben servir públicamente, estos pueden ser imágenes, archivos CSS o JS u otro recurso estático.

Mientras que si creamos un proyecto utilizando create-react-app, dentro de esta carpeta encontraremos un archivo fundamental llamado index.html que será el encargado de mostrar el contenido.

Carpeta /src

Dentro de esta carpeta es donde trabajaremos principalmente.

En ella, crearemos nuestros componentes y todos aquellos archivos personalizados que desarrollemos para mostrar información en el navegador.

Ejercicio

Continuemos con nuestro ejercicio guiado:

1. Ya que tenemos nuestra aplicación creada, veamos qué contiene el archivo **App.jsx**.
2. Generemos un componente que se llame **Título.jsx** y que muestre un texto que diga “Hola Mundo”.
3. Llamemos este componente Título en nuestro **App.jsx** para mostrarlo en el navegador.
4. Observemos la página en el navegador.
5. Al terminar de trabajar en el proyecto cerremos el servidor (ctrl + c)

Nota: export e import son palabras reservadas del lenguaje JavaScript y las utilizamos para exportar módulos, clases, funciones, variables y constantes de archivos para importarlas en los que necesitemos

{desafío}
latam_

Demostración



Ejercicio

Practiquemos lo aprendido, repitiendo los siguientes pasos:

- Crear una nueva aplicación en React desde 0.
- Levanta el servidor.
- Crear un nuevo componente llamado Prueba.jsx que muestre en pantalla un párrafo lorem ipsum.
- Desde el archivo App.jsx carga el componente nuevo.
- Revisa la página en el navegador.

Ejercicio

¡Manos al teclado!



`/* Crear una aplicación y un componente utilizando React */`



`/* Crear una componente que cargue otra componente */`

`/* Crear una componente que cargue múltiples componentes */`

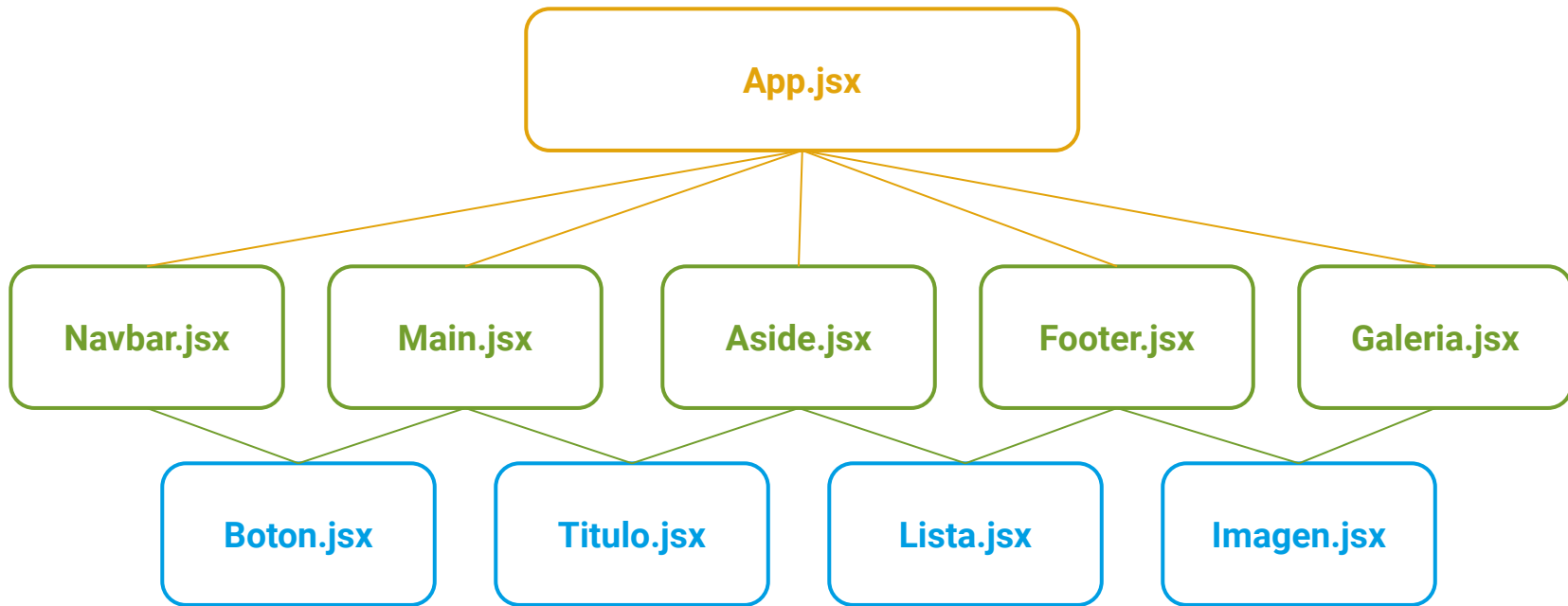
`/* Pasar props a un componente */`

Objetivos

Las componentes en React

Componentes React

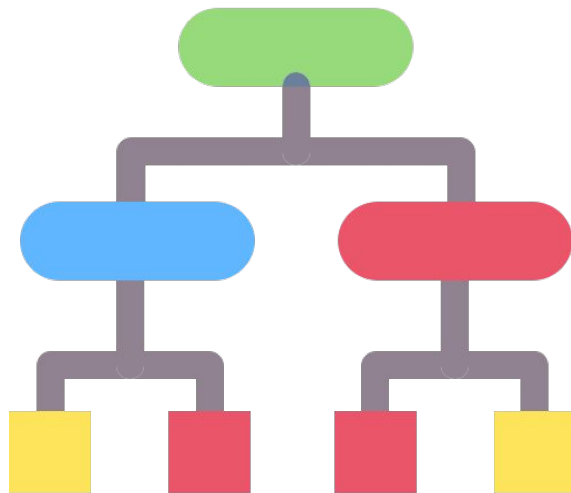
Tienen una estructura de árbol



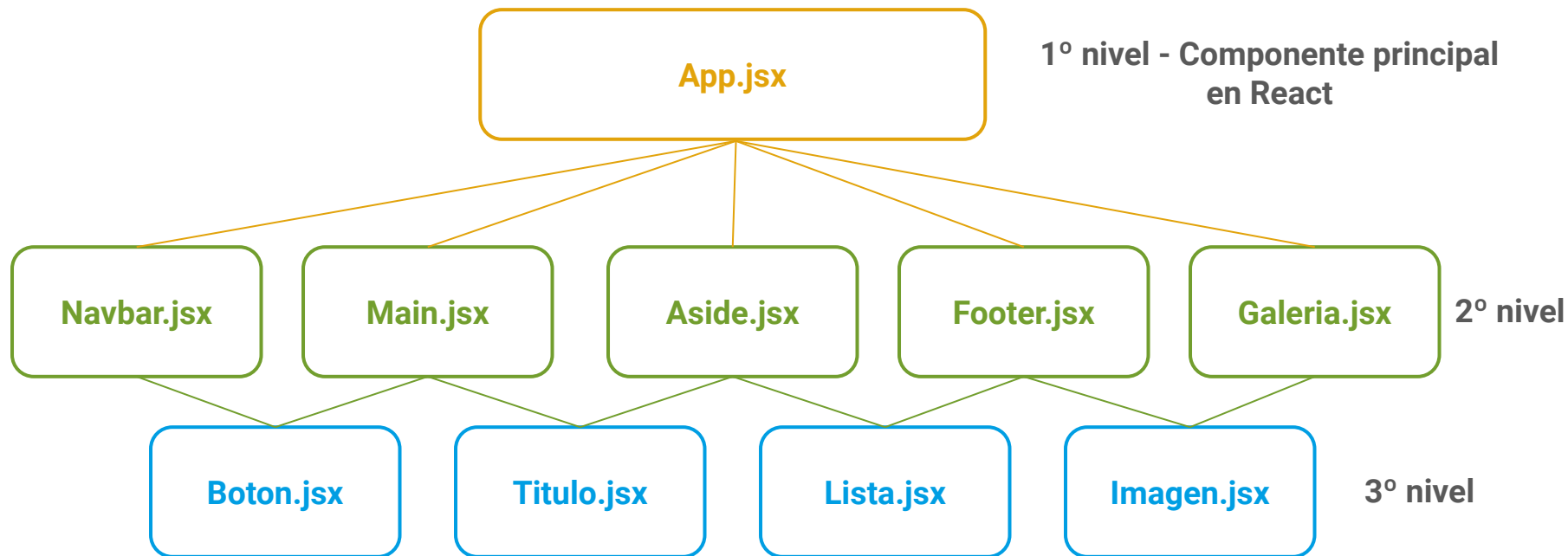
Componentes React

Tienen una estructura de árbol

- En el nivel superior tenemos el archivo principal encargado de hacer el render en el navegador de todo lo que construyamos en React.
- En los siguientes dos niveles tenemos un conjunto de componentes que, a partir de la creatividad y lógica de programación, arman la estructura de nuestra aplicación web.

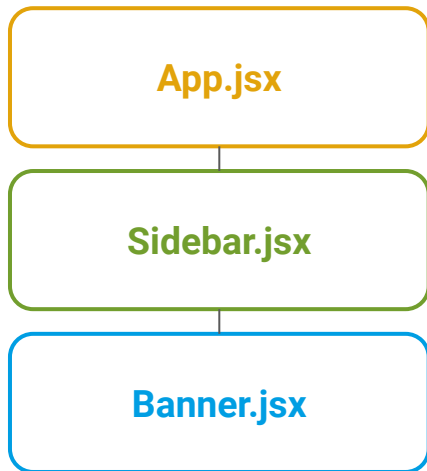


Componentes React

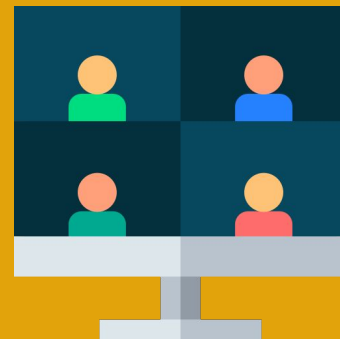


Realicemos un ejercicio guiado:

- Crea el proyecto **arbol-de-componentes**
- Genera el componente Sidebar.jsx
- Importa el componente Sidebar en el componente principal App.jsx
- Crea el componente Banner.jsx con un texto.
- Importa el componente Banner desde el sidebar.



Demostración



**Podemos devolver múltiples componentes
siempre y cuando las envolvamos en un
componente padre**



Esto es un error



```
import Producto from './Producto'  
  
function App() {  
  return (  
    <Producto />  
    <Producto />  
    <Producto />  
  )  
}
```



¡Esto está OK!



```
import Producto from './Producto'  
  
function App () {  
  return (  
    <div>  
      <Producto />  
      <Producto />  
      <Producto />  
    </div>  
  )  
}
```


En un proyecto nuevo llamado tienda:

- Crear un componente llamado Producto.jsx
 - El producto debe tener un nombre y un precio.
- Importar el producto desde app.
- Utilizarlo múltiples veces en el JSX



Hint: Recuerda envolver los componentes en el div

Ejercicio

¡Manos al teclado!



/ Crear una aplicación y un componente utilizando React */*



/ Crear un componente que cargue otro componente */*



/ Crear un componente que cargue múltiples componentes */*



/ Pasar props a un componente */*

Objetivos

Propiedades (props)



Los **props** son valores que puede recibir un componente, y nos sirven para personalizarlos y utilizarlos para distintas situaciones.

Props

```
const Producto = (props) => {  
  return <h1> {props.nombre} </h1>;  
};  
  
export default Producto;
```

Para crear un producto con props, debemos especificar que nuestra función recibe como argumento un objeto.

Podemos utilizar javascript dentro de las llaves en JSX.
El resultado de la expresión se muestra en el HTML.

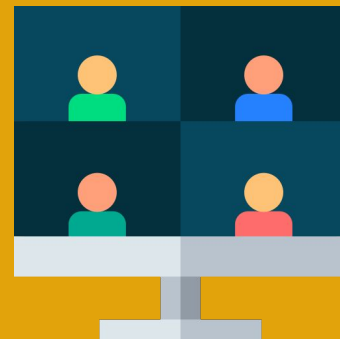
```
<Producto nombre="Mi primer producto"/>
```

Al crear un producto, pasamos los props como atributos de HTML.

Realicemos un ejercicio guiado:

1. Al proyecto tienda, agregaremos el prop nombre y precio al componente de producto
2. Modificaremos el jsx para que devuelva el nombre junto al precio.
3. Desde el archivo app, agregaremos y daremos valores a los productos creados.
4. Veamos los productos creados desde el navegador.

Demostración



En el proyecto tienda:

- Agregar los props de stock y descripción a los productos.
- Incluir estos elementos en el JSX devuelto.
- Pasar los props al producto en el archivo App.jsx

Ejercicio

¡Manos al teclado!



/ Crear una aplicación y un componente utilizando React */*



/ Crear una componente que cargue otra componente */*



/ Crear una componente que cargue múltiples componentes */*



/ Pasar props a un componente */*



Objetivos



Cierre



¿Existe algún concepto que no
hayas comprendido?

Reflexionemos

- Revisemos las temáticas a revisar en el material de estudio asincrónico:
 - Expresiones matemáticas en JSX
 - Destructuring de props
 - Valores por defecto de props
 - Renders condicionales
 - CSS y JSX
 - Instalación de componentes adicionales como bootstrap
- Revisemos el desafío de la unidad leyendo la descripción y requerimientos.

¿Qué sigue?

¿Tienen alguna duda respecto al Desafío?



*Academia de
talentos digitales*

www.desafiolatam.com



/DesafioLatam



/DesafioLatam



/DesafioLatam



/DesafioLatam